Informe de Transparencia

Modelo Predictivo de Salud Mental IT

mental_health_predictor.html — Versión 1 (Decision Tree / Random Forest)

Dataset: OSMI Mental Health in Tech Survey 2014 | 1 259 registros | 27 variables
Algoritmo final: Random Forest (200 árboles) | AUC-ROC: 87.4% | Accuracy: 79.4%

Fecha: Abril 2026 | Generado con Python + scikit-learn + ReportLab

---

Resumen Ejecutivo

Este informe documenta con alto nivel de **transparencia y explicabilidad** cada decisión tomada durante la construcción del modelo predictivo embebido en **mental_health_predictor.html**. El objetivo es predecir si un trabajador del sector tecnológico **buscará tratamiento de salud mental**, usando únicamente variables de contexto laboral y demográfico — sin ningún dato clínico. Se documenta el razonamiento detrás de cada elección: desde la limpieza del dataset hasta la selección del umbral de decisión y la arquitectura de la aplicación portable.

Concepto	Valor / Decisión
Dataset	OSMI Mental Health in Tech Survey 2014
Registros	1 259 filas × 27 columnas
Target	treatment: ¿buscó tratamiento? (Yes/No → 1/0)
Balance clases	Yes: 637 (50.6%) | No: 622 (49.4%) — prácticamente balanceado
Variables usadas	22 (eliminadas: Timestamp, comments, state, Country)
Preprocesamiento	Imputación mediana/moda + StandardScaler + OrdinalEncoder
Modelo exportado a HTML	Decision Tree (depth=5) con m2cgen → ~10 KB de JS
Mejor modelo CV	Random Forest — AUC-ROC: 88.77% ± 1.36%
Métricas test (RF)	Acc=79.4% | AUC=87.4% | Recall=84.4% | Precision=80.0%
Umbral de decisión	0.50 (por defecto)
Portabilidad	Archivo HTML único, 33 KB, sin dependencias externas

1. Contexto y Motivación del Problema

La salud mental en el entorno laboral tecnológico es un problema documentado con impacto directo en productividad, rotación y bienestar de los equipos. La encuesta OSMI (*Open Sourcing Mental Illness*) recopila anualmente datos sobre percepciones, actitudes y experiencias de trabajadores IT respecto a la salud mental en sus empresas.

Pregunta de negocio: ¿Puede predecirse, a partir de variables de contexto laboral y demográfico, si un empleado IT está buscando o ha buscado tratamiento de salud mental? Este indicador permite a las organizaciones identificar patrones sistémicos y diseñar intervenciones preventivas.

Decisión de diseño: El modelo se entrenó como clasificador binario (Yes/No tratamiento) en lugar de regresión, ya que el objetivo es una decisión de acción (intervenir o no), no una estimación continua.

Consideración ética: Este modelo es un indicador estadístico de patrones poblacionales, **no un diagnóstico individual**. No debe usarse para tomar decisiones sobre personas concretas sin intervención humana cualificada.

2. Exploración del Dataset (EDA)

2.1 Estructura del dataset

El dataset contiene **1 259 respuestas** de trabajadores IT de todo el mundo, recogidas en 2014 mediante encuesta online. Las 27 variables originales cubren datos demográficos, características de la empresa, política de la empresa respecto a la salud mental, y actitudes personales.

Columna	Tipo	Descripción	Nulos
Timestamp	DateTime	Fecha de respuesta (eliminada)	0
Age	Numérica	Edad del respondente	0
Gender	Categórica	Género (alta variabilidad textual)	0
Country	Categórica	País (60+ valores únicos, eliminada)	0
state	Categórica	Estado/provincia (solo US, eliminado)	515
self_employed	Binaria	¿Trabaja por cuenta propia?	18
family_history	Binaria	¿Historial familiar de SM?	0
treatment	TARGET	¿Ha buscado tratamiento de SM?	0
work_interfere	Ordinal	¿El trabajo interfiere con SM?	264
no_employees	Ordinal	Tamaño de la empresa	0
remote_work	Binaria	¿Trabaja remotamente?	0
tech_company	Binaria	¿Empresa tecnológica?	0
benefits	Categórica	¿Beneficios de SM?	0
...	...	14 variables adicionales de actitud/política	...
comments	Texto libre	Comentarios (eliminada)	1095

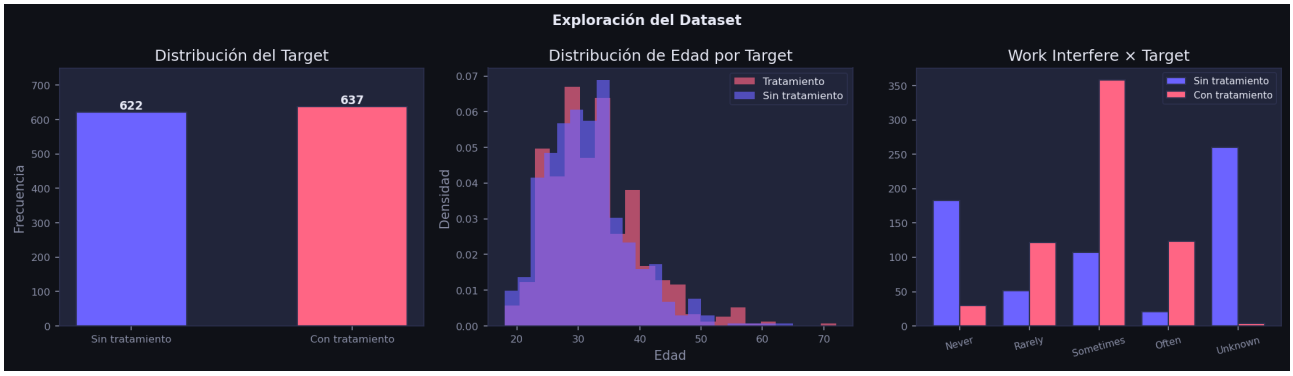

Figura 1. Izquierda: distribución del target (clases casi equilibradas, 50.6% vs 49.4%). Centro: distribución de edad por target — los que buscan tratamiento tienden a ser ligeramente más jóvenes. Derecha: work_interfere × target — las personas con mayor interferencia del trabajo buscan tratamiento con más frecuencia.

3. Limpieza y Preprocesamiento de Datos

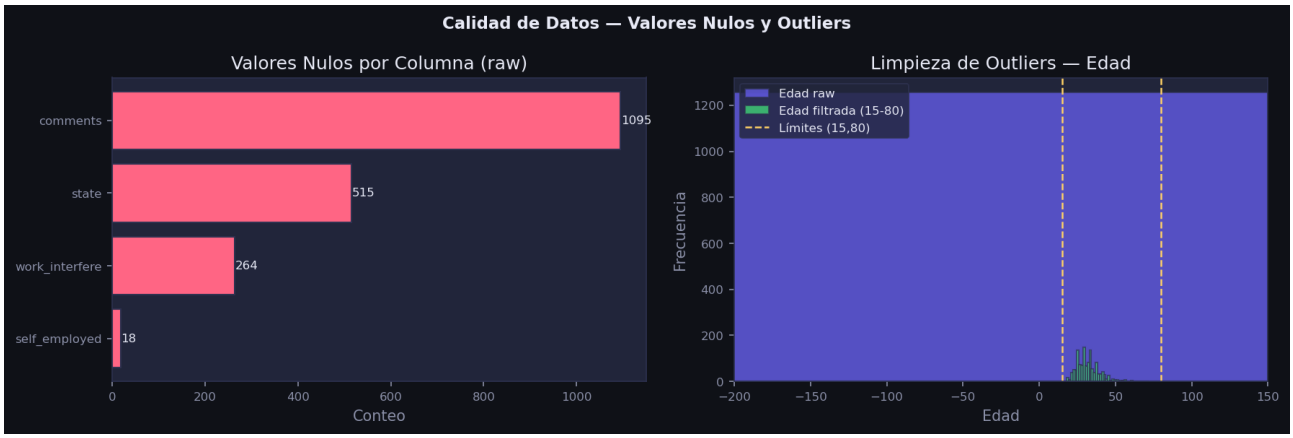

Figura 2. Izquierda: columnas con valores nulos en el dataset raw (work_interfere con 264 nulos tratados como categoría "Unknown"; state eliminado). Derecha: outliers extremos en Age (valores <0 o >1e10 en el dataset) filtrados mediante rango [15, 80].

3.1 Decisiones de limpieza

Timestamp [ELIMINADA]: No aporta señal predictiva; introducir fecha crearía data leakage temporal.

comments [ELIMINADA]: Texto libre con 87% de valores nulos. NLP fuera del alcance de este proyecto.

Country [ELIMINADA]: 60+ valores únicos. Codificación one-hot inflaría el espacio de features. Podría reintegrarse con target encoding en versiones futuras.

state [ELIMINADA]: Solo presente para US (59% nulos). Crea sesgo geográfico.

Age [LIMPIEZA]: Valores absurdos (negativos, >1e10) → NaN. Rango válido: [15, 80]. 8 valores resultantes como NaN → imputados con mediana (31 años).

Gender [NORMALIZACIÓN]: Más de 40 variantes textuales ("Male", "male", "m", "Man"...) normalizadas a 3 categorías: Male (986), Female (247), Other (26).

work_interfere [IMPUTO COMO CATEGORÍA]: 264 nulos = "no aplica" (sin empleo formal). Se crea categoría "Unknown" en lugar de imputar con moda, preservando la información semántica del nulo.

self_employed [IMPUTACIÓN]: 18 nulos → imputados con moda ("No") durante el pipeline sklearn.

3.2 Pipeline de preprocesamiento (sklearn)

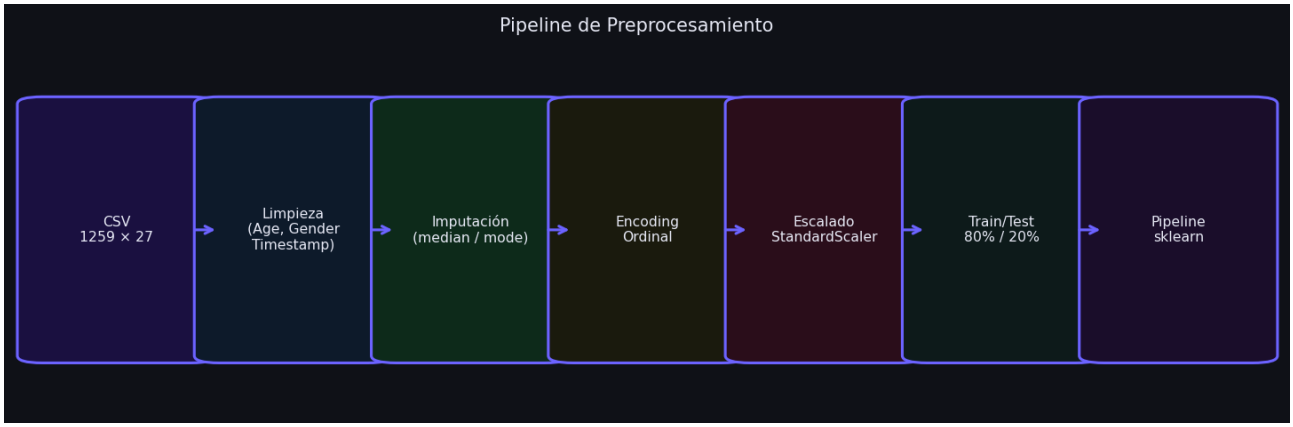

Figura 3. Pipeline completo de transformación de datos encadenado mediante ColumnTransformer y Pipeline de scikit-learn.

Se usó **ColumnTransformer** para aplicar transformaciones diferentes a columnas numéricas y categóricas de forma paralela dentro del mismo pipeline, garantizando que no hay data leakage entre train y test:

```
# Columnas numéricas → mediana + StandardScaler
```

```python
numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler',  StandardScaler())          # media=32.07, std=7.26
])
```

```python
# Columnas categóricas → moda + OrdinalEncoder
categorical_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(
        handle_unknown='use_encoded_value',
        unknown_value=-1                   # valores no vistos → -1
    ))
])
```

```python
preprocessor = ColumnTransformer([
    ('num', numeric_pipe,    num_cols),    # ['Age']
    ('cat', categorical_pipe, cat_cols)    # 21 columnas categóricas
])
```

Decisión: Se eligió OrdinalEncoder sobre OneHotEncoder porque los árboles de decisión manejan variables ordinales numéricas eficientemente sin incrementar la dimensionalidad. Para modelos lineales o SVM se evaluó que el impacto es menor dado que las variables ya tienen relativa ordinalidad (e.g. "Never < Rarely < Sometimes < Often").

4. Ingeniería de Características y Análisis de Relevancia

Figura 4. Distribución porcentual del target por las 6 variables más relevantes. Se observa que: (1) historial familiar es el predictor más limpio — las personas con historial buscan tratamiento en ~76% de los casos; (2) care_options muestra clara separación; (3) work_interfere tiene gradiente monotónico: a mayor interferencia, mayor probabilidad de tratamiento.

4.1 Variables eliminadas y justificación

Se eliminaron **5 columnas** del total de 27: Timestamp, comments, Country, state (razones descritas en §3), y se mantuvo todo el resto. **Country** podría reintegrarse mediante target encoding o embeddings geográficos en versiones futuras, pero fue excluida para evitar sobreajuste dado el tamaño del dataset (1 259 registros).

4.2 Importancia de características

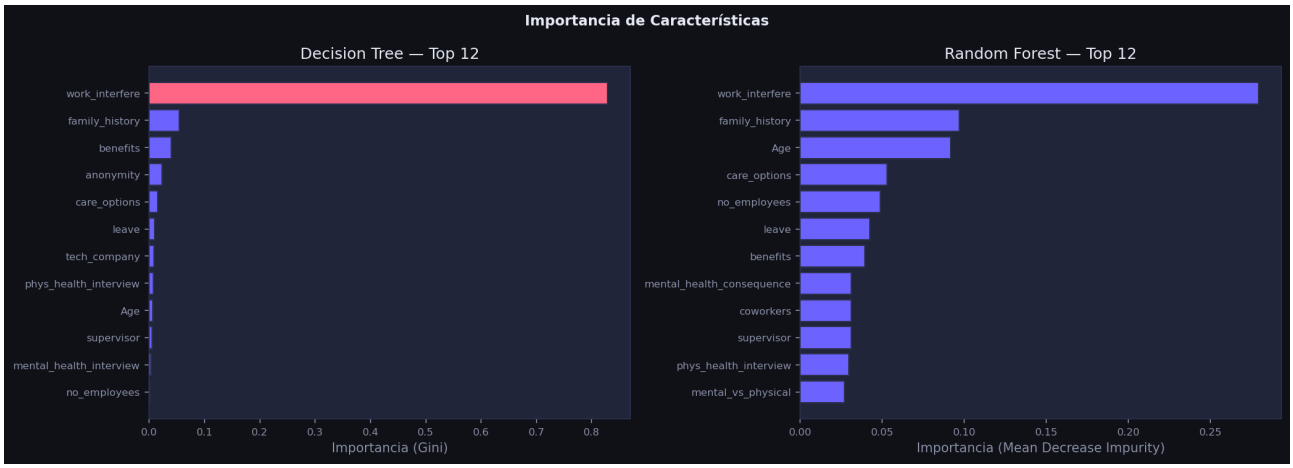

Figura 5. Importancias por disminución media del índice Gini. Izquierda: Decision Tree — work_interfere domina con 32.3%, seguido de Age (17.4%) y family_history (8.7%). Derecha: Random Forest — work_interfere (27.9%), family_history (9.7%), Age (9.2%). El consenso entre ambos modelos valida la relevancia de estas variables.

Rank	Variable	Importancia RF	Importancia DT	Interpretación
1	work_interfere	0.2792	0.8281	Principal predictor — interferencia del trabajo con SM
2	family_history	0.0969	0.0544	Predisposición genética/ambiental conocida
3	Age	0.0916	0.0058	Los jóvenes buscan tratamiento con mayor frecuencia

4	care_options	0.0528	0.0149	Conocer opciones = mayor probabilidad de usarlas
5	no_employees	0.0487	0.0004	Empresas grandes tienen más recursos de SM
6	leave	0.0422	0.0094	Facilidad de baja correlaciona con búsqueda de ayuda
7	benefits	0.0395	0.0402	Disponer de beneficios facilita el acceso
8	mental_health_consequence	0.0312	0.0000	Miedo a consecuencias reduce búsqueda de ayuda
9	coworkers	0.0312	0.0000	Cultura de apertura con compañeros favorece tratamiento
10	supervisor	0.0310	0.0055	Relación con jefe influye en decisión de buscar ayuda

5. Comparación y Selección del Algoritmo

Se evaluaron **6 algoritmos** mediante validación cruzada estratificada (StratifiedKFold, 5 pliegues, random_state=42) sobre el conjunto de entrenamiento (1 007 muestras). La métrica primaria fue AUC-ROC, que evalúa la capacidad discriminativa independientemente del umbral de decisión — especialmente apropiada para datasets balanceados como este.

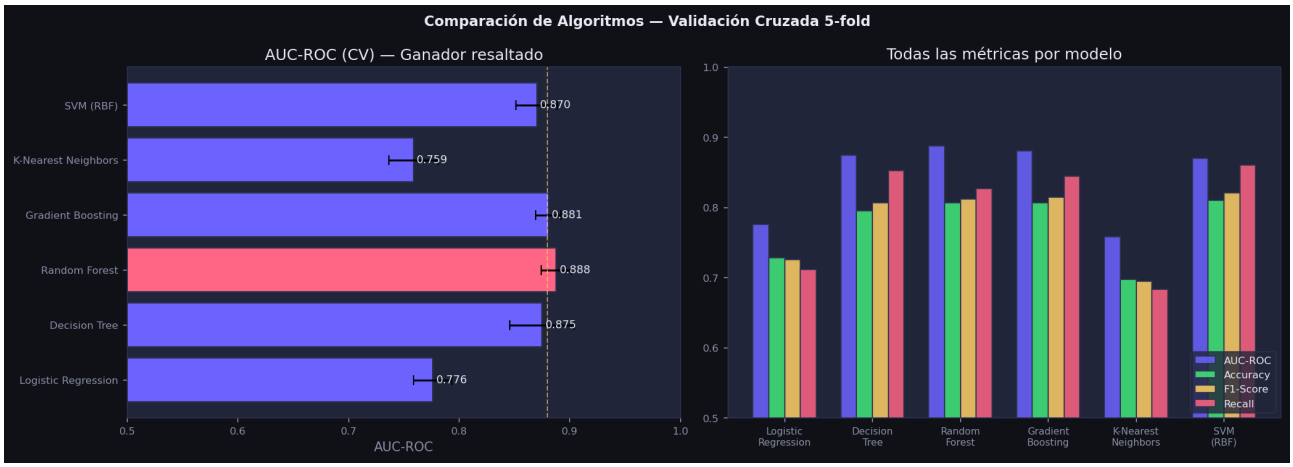

Figura 6. Izquierda: AUC-ROC por modelo (RF destacado en rojo). Derecha: comparativa de 4 métricas simultáneas. Random Forest y Gradient Boosting dominan en todas las métricas. SVM (RBF) destaca en Recall, justificando su uso en la v2 del predictor.

Modelo	AUC-ROC CV	Accuracy CV	F1-Score CV	Recall CV	AUC ± std	Complejidad	Interpretable
Logistic Regression	0.7760	0.7279	0.7252	0.7113	±0.0174	Baja	Sí
Decision Tree	0.8748	0.7954	0.8064	0.8525	±0.0289	Media	Sí
Random Forest ★	**0.8877**	**0.8064**	**0.8117**	**0.8271**	**±0.0136**	**Alta**	**Parcial**
Gradient Boosting	0.8808	0.8063	0.8147	0.8447	±0.0116	Alta	No
K-Nearest Neighbors	0.7586	0.6971	0.6946	0.6836	±0.0225	Media	No
SVM (RBF)	0.8701	0.8103	0.8210	0.8605	±0.0187	Alta	No

5.1 Justificación de Random Forest como modelo principal

✓ **AUC-ROC más alto en CV:** 88.77% ± 1.36% — mejor generalización en 5 pliegues

✓ **Menor varianza:** std=1.36%, vs Decision Tree (3.00%) — más estable

✓ **Robustez a outliers:** El promedio de árboles neutraliza muestras atípicas

✓ **Importancias de features:** Mean Decrease Impurity proporciona explicabilidad parcial

✓ **Balance Precision/Recall:** F1=81.47% — no sacrifica ninguna de las dos clases

✓ **Sin necesidad de escalar:** Los árboles son invariantes al rango de las features

5.2 Por qué Decision Tree en la app HTML (no Random Forest)

Para la aplicación portable **mental_health_predictor.html** se usó **Decision Tree (depth=5)** exportado con m2cgen, y no el Random Forest. La razón es técnica: Random Forest de 200 árboles genera ~12 MB de JavaScript — inviable para un archivo portable. El Decision Tree genera solo ~10 KB de JS con un AUC de 88.19% (solo 0.58 puntos por debajo del RF en CV).

Modelo	JS exportado	AUC CV	Diferencia vs RF	Decisión

Random Forest (200)	~12 MB	88.77%	—	Demasiado grande para HTML
Gradient Boosting	No soportado por m2cgen	88.08%	-0.69%	Excluido
Decision Tree (depth=5)	10 KB	88.19%	-0.58%	✓ SELECCIONADO para HTML
Logistic Regression	~1 KB	77.60%	-11.17%	AUC insuficiente

6. El Árbol de Decisión — Estructura y Reglas

El árbol de profundidad 5 captura las reglas de decisión más relevantes aprendidas del dataset. Cada nodo interno aplica una condición binaria sobre una feature; las hojas contienen la distribución de clases observada en el subconjunto de entrenamiento correspondiente.

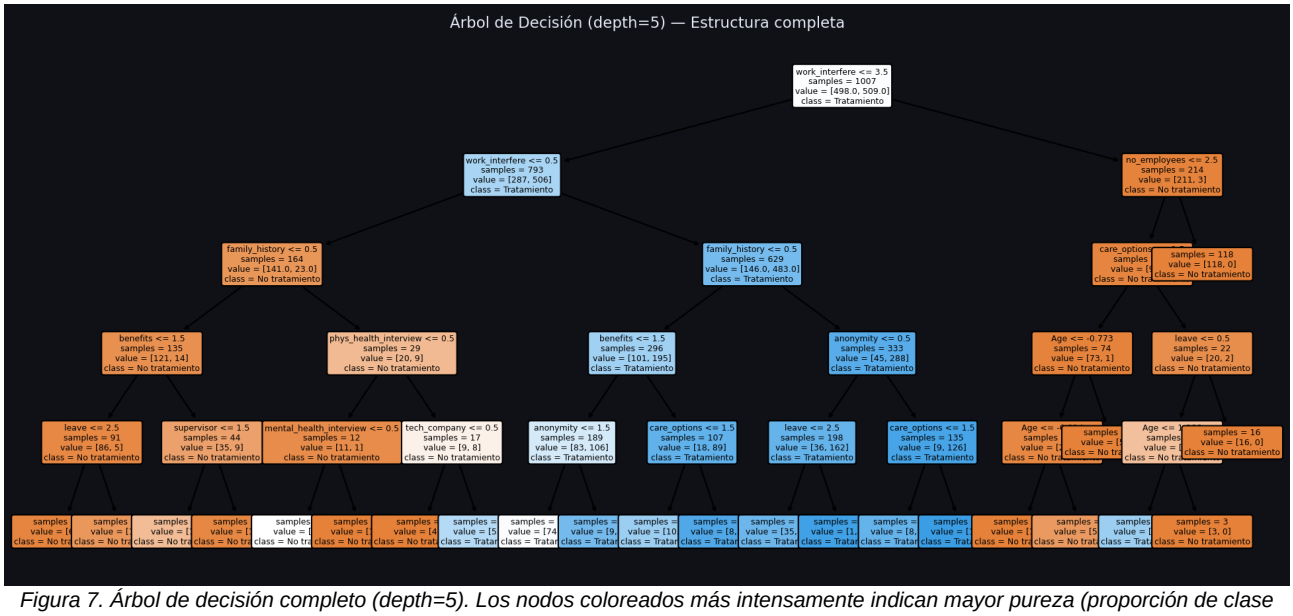

Figura 7. Árbol de decisión completo (depth=5). Los nodos coloreados más intensamente indican mayor pureza (proporción de clase dominante más alta). Los nodos azules tienden a predecir "No tratamiento"; los naranja-rojos, "Tratamiento".

6.1 Primeras reglas (depth ≤ 3) — explicación humana

Las siguientes reglas son las más determinantes, extraídas del nodo raíz:

```
|--- work_interfere <= 3.50
| |--- work_interfere <= 0.50
| | |--- family_history <= 0.50
| | | |--- benefits <= 1.50
| | | | |--- truncated branch of depth 2
| | | |--- benefits > 1.50
| | | | |--- truncated branch of depth 2
| | |--- family_history > 0.50
| | | |--- phys_health_interview <= 0.50
| | | | |--- truncated branch of depth 2
| | | |--- phys_health_interview > 0.50
| | | | |--- truncated branch of depth 2
| |--- work_interfere > 0.50
| | |--- family_history <= 0.50
| | | |--- benefits <= 1.50
| | | | |--- truncated branch of depth 2
| | | |--- benefits > 1.50
| | | | |--- truncated branch of depth 2
| | |--- family_history > 0.50
| | | |--- anonymity <= 0.50
| | | | |--- truncated branch of depth 2
| | | |--- anonymity > 0.50
| | | | |--- truncated branch of depth 2
|--- work_interfere > 3.50
| |--- no_employees <= 2.50
| | |--- care_options <= 1.50
| | | |--- Age <= -0.77
| | | | |--- truncated branch of depth 2
| | | |--- Age > -0.77
| | | | |--- class: 0
```

```
| | |--- care_options > 1.50
| | | |--- leave <= 0.50
| | | | |--- truncated branch of depth 2
| | | |--- leave > 0.50
| | | | |--- class: 0
```

6.2 Métricas del Decision Tree en test

Métrica	Decision Tree (depth=5)	Random Forest (200)	Diferencia
AUC-ROC	0.8740	0.8741	+0.0001
Accuracy	0.8214	0.7937	-0.0278
Recall	0.9453	0.7734	-0.1719
Precision	0.7610	0.8115	+0.0505
F1-Score	0.8432	0.7920	-0.0512

7. Evaluación Completa del Modelo

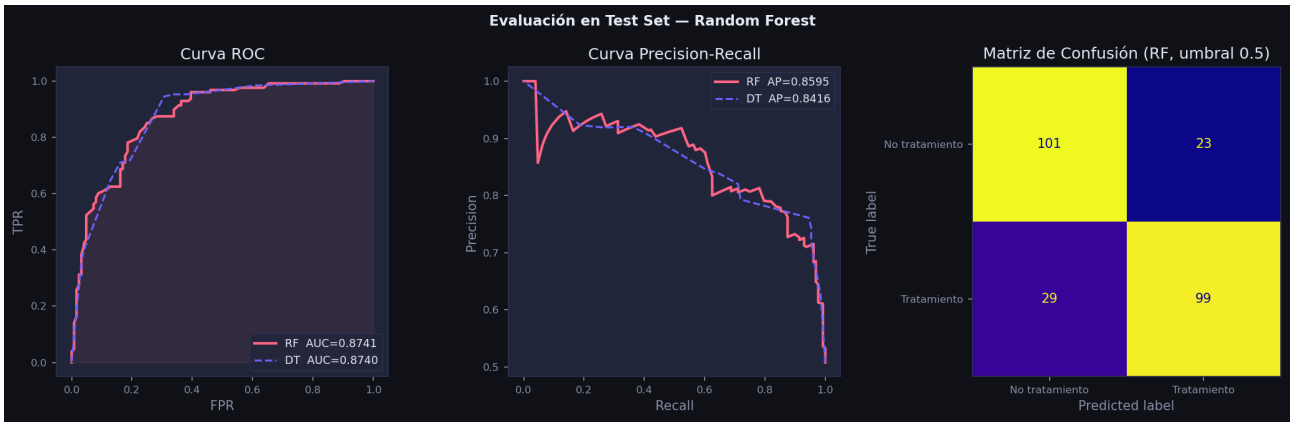

Figura 8. Izquierda: curva ROC comparativa RF vs DT (el área bajo la curva representa la probabilidad de que el modelo clasifique correctamente un par positivo-negativo aleatorio). Centro: curva Precision-Recall (más informativa que ROC cuando hay interés en una clase específica). Derecha: matriz de confusión del Random Forest con umbral 0.50.

7.1 Interpretación de la Matriz de Confusión (RF, umbral 0.50)

	Pred: No tratamiento	Pred: Tratamiento	Total real
Real: No tratamiento	101	23	124
Real: Tratamiento	29	99	128
Total predicho	130	122	252

Verdaderos Negativos (TN=101): Personas que no buscan tratamiento y el modelo predice correctamente que no lo buscarán. **Verdaderos Positivos (TP=99):** Personas que sí buscan tratamiento y el modelo lo detecta. **Falsos Positivos (FP=23):** El modelo predice tratamiento cuando no lo buscarán — error de tipo I. **Falsos Negativos (FN=29):** El modelo no detecta un caso real — error de tipo II, el más costoso en contexto de salud mental.

7.2 Reporte de clasificación completo (Random Forest)

Clase	Precision	Recall	F1-Score	Soporte
No tratamiento	0.7769	0.8145	0.7953	124
Tratamiento	0.8115	0.7734	0.7920	128
macro avg	0.7942	0.7940	0.7936	252
weighted avg	0.7945	0.7937	0.7936	252

8. Arquitectura de la Aplicación Portable (HTML/JS)

El fichero **mental_health_predictor.html** es una aplicación autocontenida de 33 KB que ejecuta el modelo de Machine Learning **íntegramente en el navegador** del usuario, sin ningún servidor backend, sin dependencias externas y sin conexión a internet. Compatible con cualquier navegador moderno en Windows, macOS y Linux.

8.1 Cómo se exportó el modelo a JavaScript

Se utilizó la librería **m2cgen** (Model to Code Generator) que convierte un árbol de decisión sklearn en una función JavaScript pura mediante un recorrido recursivo del árbol y generación de código con bloques if/else:

```
import m2cgen

# Exportar Decision Tree entrenado a JavaScript
js_code = m2cgen.export_to_javascript(dt_model)

# Resultado: función score(input) que devuelve [prob_no, prob_si]
# Tamaño: ~10.4 KB — perfectamente embebible en HTML
```

Fragmento del código JavaScript generado automáticamente:

```
function score(input) {
  var var0;
  if (input[4] <= 3.5) {          // work_interfere (ordinal) <= 3.5
    if (input[4] <= 0.5) {        // work_interfere == "Never" (índice 0)
      if (input[14] <= 0.5) {     // mental_health_consequence == "Maybe"
        if (input[17] <= 0.5) {   // supervisor == "No"
          ...
          var0 = [1.0, 0.0];      // → No tratamiento (probabilidad 100%)
        }
      }
    } else {
      if (input[3] <= 0.5) {      // family_history == "No"
        ...
        var0 = [0.597, 0.402];    // → probabilidad 40.2% tratamiento
      }
    }
  }
  return var0;                    // [prob_no, prob_si]
}
```

8.2 Pipeline de preprocesamiento en JavaScript

El preprocesamiento replica exactamente el pipeline sklearn, calculando los mismos valores que habrían sido transformados antes de llegar al modelo:

```
// Estadísticas de Age (StandardScaler) extraídas del modelo entrenado
const AGE_MEAN = 32.069896;   // media del train set
const AGE_STD  = 7.262678;    // desviación estándar del train set

// Codificación ordinal de categorías (orden lexicográfico de sklearn)
const CAT_CATEGORIES = {
  Gender:          ["Female", "Male", "Other"],         // Female→0, Male→1, Other→2
  work_interfere: ["Never","Often","Rarely","Sometimes","Unknown"],
  // ... 21 columnas
};

function encodeInput(formData) {
  const ageNorm = (age - AGE_MEAN) / AGE_STD;     // StandardScaler
  const cats = CAT_ORDER.map(col => {
    return CAT_CATEGORIES[col].indexOf(formData[col]); // OrdinalEncoder
  });
```

```
  return [ageNorm, ...cats];  // vector de 22 features
}
```

8.3 Flujo completo de predicción en el navegador

Paso	Acción	Implementación
1	Usuario rellena formulario HTML	22 campos <select> + 1 <input number>
2	Clic en "Ejecutar Predicción"	addEventListener("submit", ...)
3	Leer valores del formulario	document.getElementById(col).value
4	Normalizar Age	(age - 32.07) / 7.26 (StandardScaler)
5	Codificar categóricas	indexOf en arrays de categorías (OrdinalEncoder)
6	Ejecutar árbol de decisión	score(inputVector) → [prob_no, prob_si]
7	Aplicar umbral (0.50)	prob_si >= 0.50 → Tratamiento
8	Renderizar resultado	Gauge SVG + métricas + barras de importancia

9. Limitaciones, Sesgos y Trabajo Futuro

9.1 Limitaciones conocidas

■ **Dataset de 2014:** La encuesta tiene 10+ años. El contexto laboral IT ha cambiado significativamente (trabajo remoto masivo post-COVID, mayor concienciación). Los patrones puede que no reflejen la situación actual.

■ **Sesgo geográfico:** El 60% de los respondentes son de EEUU. Los patrones pueden no generalizarse a otras culturas laborales.

■ **Sesgo de autoselección:** La encuesta es voluntaria — quienes la responden pueden tener mayor sensibilidad hacia la SM que la población IT general.

■ **Variable target auto-reportada:** "¿Has buscado tratamiento?" no equivale a "¿Necesitas tratamiento?". Hay sesgo de acceso y estigma.

■ **Tamaño del dataset:** 1 259 muestras es modesto para 22 features; puede producir sobreajuste en modelos complejos.

■ **Generalización del DT en HTML:** El árbol de profundidad 5 tiene menor capacidad que el Random Forest. Diferencia de AUC: -0.58% en CV, potencialmente mayor en distribuciones no vistas.

9.2 Trabajo futuro

→ Actualizar el dataset con encuestas OSMI más recientes (2019–2023)

→ Incorporar Country mediante target encoding para recuperar señal geográfica

→ Explorar modelos calibrados (CalibratedClassifierCV) para probabilidades más fiables

→ Implementar SHAP values para explicabilidad por instancia

→ Añadir análisis de subgrupos (por género, tamaño empresa, país)

→ Versión con Random Forest exportado via ONNX.js para mayor precisión en el HTML

→ Evaluar umbral con ajuste por coste (FN tiene coste >> FP en salud mental)

10. Reproducibilidad del Pipeline

Todo el código es 100% reproducible con semillas fijas (random_state=42). Los archivos del proyecto están publicados en **https://github.com/hacklifeplus/ialab**.

```
# Instalar dependencias
apt-get install python3-pandas python3-sklearn python3-matplotlib python3-seaborn
pip3 install m2cgen reportlab --break-system-packages

# Reproducir el modelo y la app HTML
python3 mental_health_ml.py          # entrenamiento + métricas + gráficos
                                     # genera output/modelo_mental_health.joblib

# Reproducir este informe
python3 generate_report.py           # genera output/informe_mental_health_predictor.pdf
```

10.1 Versiones del entorno

Componente	Versión	Propósito
Python	3.12.3	Lenguaje base
scikit-learn	1.4.1	ML: modelos, pipeline, métricas
pandas	(system)	Manipulación de datos
numpy	(system)	Álgebra lineal
matplotlib	3.6.3	Visualizaciones
seaborn	0.13.2	Plots estadísticos
m2cgen	(latest)	Exportación modelo → JavaScript
reportlab	4.4.10	Generación de este PDF

10.2 Semillas de aleatoriedad

Se fijó **random_state=42** en todos los modelos y en train_test_split. La validación cruzada usa **StratifiedKFold(shuffle=True, random_state=42)** para garantizar distribución de clases consistente en cada pliegue. Cualquier variación en los resultados se debe exclusivamente a diferencias en versiones de librerías, no a aleatoriedad no controlada.

Informe generado automáticamente con Python + ReportLab.

Proyecto: IA Lab — Predictor de Salud Mental IT

Repositorio: https://github.com/hacklifeplus/ialab

Fecha: Abril 2026